

SET1 A4. Разные алгоритмы решения одной задачи

1. Algorithm1 – ищет элемент, встречающийся более $n/2$ раз, перебирая все пары индексов и подсчитывая частоту каждого элемента. Если найден элемент с частотой $> n/2$, возвращает его, иначе возвращает элемент с максимальной частотой (даже если он не удовлетворяет условию).
2. Algorithm2 - ищет элемент, встречающийся более $n/2$ раз, проходит по массиву, поддерживая текущего кандидата и счетчик. Если встречаются такой же элемент - счетчик увеличивается, в противном случае - уменьшается, когда счетчик = 0 - кандидатом становится текущий элемент, но проблема в том, что алгоритм не проверяет действительно ли кандидат подходит под условие
3. Algorithm3 - сначала сортирует массив, затем в одном проходе подсчитывает подряд идущие одинаковые элементы, если длина превышает $n/2$, возвращает этот элемент. Однако после окончания цикла не проверяется последняя накопленная группа

ВЫВОД: все три алгоритма решают задачу поиска элемента, встречающийся более $n/2$ раз, но alg1 может вернуть неверный результат при отсутствии подходящего элемента, alg2 возвращает кандидата, но без подтверждения, что он действительно подходит, alg3 может пропустить элемент из-за незавершенной проверки после цикла. Следовательно, утверждение, что алгоритмы выдают всегда одинаковый результат, ложно.

Рассмотрим несколько примеров:

1. ПРИМЕР №1: $A = [6, 6, 8, 6]$, $n = 4$, предполагаемый результат = 6:

Algorithm1

$c = 0$, $ind = -1$

$i = 0$ ($A[0] = 6$):

$c1 = 0$

$j=0$: $A[0]=6$, $A[0]=6 \rightarrow c1=1$

$j=1$: $A[0]=6$, $A[1]=6 \rightarrow c1=2$

$j=2$: $A[0]=6$, $A[2]=8 \rightarrow$ нет

$j=3$: $A[0]=6$, $A[3]=6 \rightarrow c1=3$

$c1=3 > c=0 \rightarrow c=3$, $ind=0$

$c=3 > n/2=2$? Да $\rightarrow$ return $A[0] = 6$

Результат: 6

Algorithm2

$c = 1$, $ind = 0$ (кандидат $A[0]=6$)

$i=1$: $A[1]=6 = A[ind]=6 \rightarrow c=2$

$i=2$: $A[2]=8 \neq 6 \rightarrow c=1$

$i=3$: $A[3]=6 = 6 \rightarrow c=2$

Результат: 6

Algorithm3

Сортируем массив: $A = [6, 6, 6, 8]$

$c = 1$ (для $A[0]=6$)

$i=1$: $A[1]=6 = A[0]=6 \rightarrow c=2$

$i=2$: $A[2]=6 = A[1]=6 \rightarrow c=3$

$i=3$: $A[3]=8 \neq A[2]=6 \rightarrow$ Проверка: $c=3 > n/2=2$? Да $\rightarrow$ return $A[2]=6$.

Результат: 6

ИТОГ: Все три алгоритма вернули 6.

2. ПРИМЕР №2: A[1, 2, 3, 4], n = 4

Algorithm1

c=0, ind=-1

i=0 (A[0]=1): c1=1 (только j=0) → c=1, ind=0

i=1 (A[1]=2): c1=1 → c остаётся 1 (не больше)

i=2 (A[2]=3): c1=1 → без изменений

i=3 (A[3]=4): c1=1 → без изменений

После цикла: c=1, ind=0.

Условие $c > n/2$? $1 > 2$? Нет.

Результат: нет возврата (ошибка)

Algorithm2

c=1, ind=0 (кандидат A[0]=1)

i=1: A[1]=2 ≠ 1 → c=0 → c=0 → ind=1, c=1 (кандидат A[1]=2)

i=2: A[2]=3 ≠ 2 → c=0 → ind=2, c=1 (кандидат A[2]=3)

i=3: A[3]=4 ≠ 3 → c=0 → ind=3, c=1 (кандидат A[3]=4)

Возвращает A[3]=4 (без проверки).

Результат: 4 (ошибочно)

Algorithm3

Сортируем: A = [1, 2, 3, 4]

c=1

i=1: 2 ≠ 1 → c=1 > 2? Нет → c=1 (теперь для элемента 2)

i=2: 3 ≠ 2 → c=1 > 2? Нет → c=1 (для элемента 3)

i=3: 4 ≠ 3 → c=1 > 2? Нет → c=1 (для элемента 4)

После цикла нет кода — функция не возвращает значения (ошибка).

Результат: нет возврата (ошибка)

ИТОГ: Результаты могут отличаться.

Асимптотическая временная сложность:

- Algorithm1:

Два вложенных цикла по n → $O(n^2)$

- Algorithm2:

Один проход по массиву → $O(n)$

- Algorithm3:

Сортировка (sort(A)) — в худшем случае $O(n \log n)$, затем один проход $O(n)$

Итого: $O(n \log n)$

Доработка алгоритмов:

Algorithm1:

if $c > n/2$

return A[ind]

else

return NULL

Пояснение: Предложенная доработка корректна, т.к. алгоритм уже находит элемент с максимальной частотой c и его индекс ind . Добавленная проверка после завершения циклов гарантирует, что возвращаемое значение будет подходящим элементом (если $c > n/2$) или NULL (если такого элемента нет), что полностью соответствует постановке задачи.

Algorithm2

```
count = 0
for i = 0 to n-1
    if A[i] = A[ind]
        count = count + 1
if count > n / 2
    return A[ind]
else
    return NULL
```

Пояснение: Предложенная доработка корректна, т.к. алгоритм гарантированно находит кандидата, но не проверяет его фактическую частоту. Добавленный второй проход для подсчёта вхождений кандидата позволяет убедиться, что он действительно встречается больше $n/2$ раз, что исключает ложные возвращения при отсутствии нужного элемента.

Algorithm3

```
if c > n / 2
    return A[n-1]
else
    return NULL
```

Пояснение: Предложенная доработка корректна, т.к. после сортировки возвращаемый элемент (если существует) образует непрерывную группу. Проверка счётчика c после завершения цикла обеспечивает учёт последней группы элементов в массиве, которая в противном случае могла бы быть пропущена. Это гарантирует, что элемент будет возвращён тогда и только тогда, когда его частота превышает $n/2$.

Влияние доработок на сложность

- Algorithm1:
Добавлена проверка после двойного цикла — $O(1)$ или $O(n)$. Если пересчёт — $O(1) + O(n^2) = O(n^2)$ / $O(n) + O(n^2) = O(n^2)$ (не ухудшает)
- Algorithm2:
Добавлен второй проход $O(n)$ для проверки кандидата. Было $O(n)$, стало $O(n) + O(n) = O(n)$ (не ухудшает, т.к. $O(n)$ остаётся)
- Algorithm3:
Добавлена проверка после цикла $O(1)$. Было $O(n \log n)$, осталось $O(n \log n)$ (не ухудшает)

ВЫВОД: верхние границы не изменились.